

Programming Language Principles, Design, and Implementation

Compilers - Introduction

Vincent Rahli

University of Birmingham

Where are we?

- ▶ Part 1: Principles of programming
- ▶ **Part 2: Compilers**
- ▶ Part 3: Computer-aided verification

Today

Introduction

- ▶ Motivation
- ▶ Structure of a compiler

Further reading:

- ▶ Chapter 1 of “Modern Compiler Implementation”
- ▶ Chapter 1 of the Dragon book (“Compilers – Principles, Techniques, and Tools”)

Introduction

A **compiler** is a **language processor** that translates a program written in some programming language (**the source language**) into code that can be executed by a computer (**the target language**).

It is in charge of

- ▶ generating a target program **“equivalent”** to the source program
- ▶ reporting **errors** detecting during the translation

There are other kinds of programming language processors, such as **interpreters**. As opposed to an interpreter that directly executes the operations of the source program, a compiler generates a target program, which itself needs to be executed. Trade-offs:

- ▶ machine-language target programs are usually faster
- ▶ interpreters can usually give better error messages

Introduction

The first electronic computers appeared in the 40s, and were programmed in **machine language**.

- ▶ very low level
- ▶ slow & tedious programming
- ▶ error prone
- ▶ hard to understand and maintain

There was a need for **friendlier programming languages** to solve the above issues

- ▶ mnemonic assembly languages in the 50s
- ▶ Fortran (1957), Cobol (1959), Lisp (1958) in the 50s

Many more programming languages were introduced in the following decades, which are more efficient, easier to use, higher-level, etc.

Introduction

New demands from both sides:

- ▶ The advances in **programming languages** led to advances in **compilers**: need to support new language features
- ▶ The advances in **computer architectures** also led to advances in **compilers**: take advantage of new hardware capabilities

Writing a compiler is challenging. It must bridge the gap between a **high-level programming language** and the computer, while

- ▶ generating efficient machine code
- ▶ being efficient itself
- ▶ generating semantically equivalent code
- ▶ handling several target machines

Introduction

Compilers rely on concepts and techniques that are widely applicable.

They make use of standard **mathematical models** such as **finite-state machines** and **trees**, and corresponding algorithms.

Building a compiler is not only a **science** but also an **art**.

- ▶ mathematical tools such as **graphs** and **matrices** are used to optimize code
- ▶ however, the problem of generating optimal code from a source program is **undecidable**
- ▶ some phases, e.g., code generation and optimization, rely on **heuristics**

Optional reading: “Evolution of the Major Programming Languages”

Structure of a Compiler

Some programming languages are preprocessed using a **preprocessor**, which for example expands macros (e.g., C).

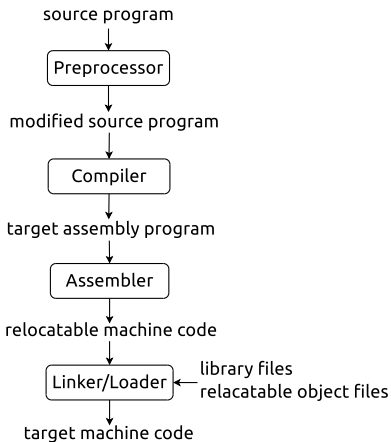
Often compilers produce **assembly** programs, which are then processed by the **assembler**, which generates machine code.

A **linker** can link multiple object files containing machine code, by resolving external memory addresses.

A **loader** puts all the object files into memory for execution.

We will see how we can generate simple assembly code from a C-like programs.

Structure of a Compiler



Structure of a Compiler

The compiler itself can be divided into multiple parts.

The **analysis** part (**frontend**)

- ▶ reads the source code
- ▶ analyzes that code
- ▶ generates some intermediate representation of that code.

Errors can be reported at any stage.

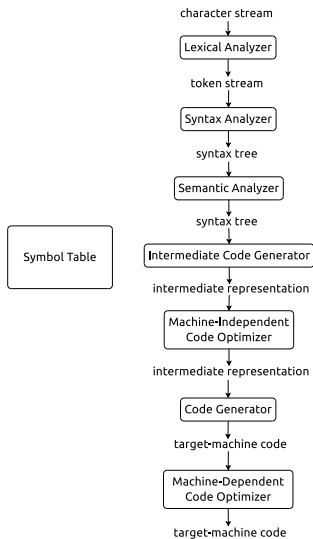
Information about the program gathered during that phase (e.g., type information) is stored into a **symbol table**, which is used in every stage.

The **synthesis** part (**backend**)

- ▶ turns the intermediate representation into a target program
- ▶ making use of the symbol table

Each part is divided into **phases**, which (typically) inputs a representation of the program, and outputs another representation of the program, getting closed to machine code.

Structure of a Compiler



Frontend

Lexical analysis. Reads the stream of characters making up the source code, and groups characters into **lexemes**, and produces **tokens** (pairs of a name and a value). Information about those tokens is stored in the **symbol table**.

Syntax analysis (or **parsing**). Reads the stream of tokens, and produces a tree representation of the program called a **syntax tree**.

Semantic analysis. Inputs the syntax tree and checks that it is **semantically** correct w.r.t. the language definition, e.g., by doing **type checking**. Type information is stored in the **symbol table**.

Intermediate code generation. Inputs the syntax tree, and generates a low-level representation (e.g., three address code) of the code meant to be easy to produce and easy to translate to target code.

Backend

Intermediate code optimization. Some compilers include a machine-independent code-optimization phase at the level of the intermediate representation (e.g., eliminating type castings).

Code generation. Inputs an intermediate representation, and generates a sequence of machine instructions. This typically includes selecting registers or memory locations for each variable.

Code optimization. Tries to improve the performance of the generated machine code.

Conclusion

What did we cover today?

- ▶ Structure of a Compiler

Next time?

- ▶ Lexical analysis